Traffic Flow Prediction and Signal Control via Graph Attention Networks and Flow Models

Project report: leak-free per-sensor baselines and the measured gap to network-aware learning

Manas Saxena

Applied Machine Learning, Project T036

October 27, 2026

Source code and data pipeline:
https://github.com/ManasSaxena14/TrafficFlowControl_AIML-DL_T036
Interactive results dashboard (deployed): https://trafficflowamldl.streamlit.app

Abstract

Predicting congestion on a road network is a spatial as well as a temporal problem: a jam forms at a bottleneck and travels backwards along the road, so the most informative signal for a sensor may sit at its neighbours. This report tests that idea on METR-LA, 207 freeway sensors in Los Angeles sampled every five minutes, for forecasts 5 to 60 minutes ahead. We build a leak-free preprocessing pipeline (chronological purged split, outage masking, train-only scaling), six forecasting baselines (persistence, historical average, Ridge, XGBoost, Random Forest and an LSTM without a graph), and four experiments that measure what independent per-sensor models miss. XGBoost is the strongest independent model, with a test mean absolute error of 3.02, 3.51 and 4.01 mph at 15, 30 and 60 minutes. Its error is 5.9 to 6.4 times larger in the 30 minutes after a congestion onset, and the errors of road-linked sensors are correlated (0.20 at 60 minutes) while those of unconnected sensors are not. Adding hand-made neighbour speeds lowers error at every horizon, by up to 3.5% at 15 minutes, which shows that the missing information is spatial but is only partly captured by one-hop features. All numbers in this report are generated from files in the repository, and a deployed dashboard exposes the same results.

Contents

1 Introduction **3**

2 Background and related work **3**

3 Data: METR-LA **5**

4 Methodology **7**

5 Results: forecasting baselines **11**

6 Results: identifying the gap **13**

7 Software, reproducibility and deployment **17**

8 Discussion and limitations **18**

9 Conclusions and next steps **20**

A Test MAE at every horizon **21**

B Validation scores **21**

C Traceability **22**

1 Introduction

Road congestion is costly, and forecasting traffic speed a few minutes to an hour ahead is the input to routing, incident response and signal control. Traffic is a network phenomenon: when a bottleneck saturates, vehicles queue behind it and the slowdown propagates upstream. A model that forecasts each road segment independently from that segment's own history cannot represent this mechanism, whereas a model that reads the road graph can. Recent deep-learning work operationalises this with graph neural networks [9, 11, 12], but the size of the gain over strong per-segment machine-learning baselines, and where it comes from, is rarely isolated under a leak-free protocol.

This project follows a hybrid logic: first build strong segment-level baselines, then identify with evidence what they cannot represent, and only then train a network-aware model. This report covers the first two stages, the *applied machine learning* (AML) phase. The graph deep-learning stage is future work and no result for it is claimed here.

Objectives. (i) Build a preprocessing pipeline in which nothing the models are scored on can influence what they are trained on. (ii) Implement the baselines named in the project brief (XGBoost per segment or a multi-output Random Forest) and a ladder of simpler and deeper references. (iii) Evaluate all models with one shared protocol: chronological splits, equal history windows, MAE, RMSE and MAPE by forecast horizon, and peak versus off-peak. (iv) Measure, rather than assert, where independent models fail to capture upstream and downstream propagation.

Contributions.

- A reproducible leak-free pipeline with 40 automated tests, including twelve leakage tests, and a demonstration that a commonly used window-level split lets train and validation targets overlap.
- Six baselines scored identically, with test and validation tables at every horizon, an accuracy summary, and a peak versus off-peak breakdown.
- Four gap experiments: error against horizon, error after congestion onsets, residual correlation along road links, and a neighbour-information test that bridges per-sensor and graph models.
- A literature-grounded set of four hypotheses (Section 2.5), each tested and reported with its outcome (Section 8.1), and a map of the thirteen research references to their role in the project (Table 1).
- An open-source repository and a deployed interactive dashboard of all results.

Organisation. Section 2 reviews related work, Section 3 describes the data, Section 4 the methodology, Sections 5 and 6 the results, Section 7 the software and deployment, and Sections 8 and 9 discuss limitations and next steps.

2 Background and related work

This section reviews the research the project builds on; Table 1 summarises how each reference is used.

2.1 Classical and machine-learning forecasting

Traffic speed has a strong daily and weekly rhythm, so the historical average per time slot is a standard reference. Linear models (here Ridge regression, 4), tree ensembles (Random Forests, 1; gradient-boosted trees as implemented in XGBoost, 2) and recurrent networks (LSTM, 3) can use lagged speeds and time features, but each treats a sensor as an independent series. The baseline set of the widely used DCRNN study contains HA, ARIMA with a Kalman filter, VAR, SVR, a feed-forward network and an FC-LSTM [9]; gradient-boosted and random-forest regressors are not among them, so they provide a stronger independent yardstick for graph models.

2.2 Graph-based deep learning

Yu et al. [12] formulate traffic forecasting on graphs and build the model from fully convolutional structures instead of recurrent units. Li et al. [9] model traffic as a diffusion process on a directed graph and introduce the Diffusion Convolutional Recurrent Neural Network (DCRNN), which captures spatial dependency with bidirectional random walks and temporal dependency with an encoder–decoder trained with scheduled sampling; on METR-LA and PEMS-BAY it improves on the compared baselines by 12% to 15%. The authors argue that the bidirectional walk lets the model capture influence from both upstream and downstream traffic, and report that the directed-graph model outperforms an undirected variant (GCRNN) on METR-LA (MAE 2.77, 3.15 and 3.60 against 2.80, 3.24 and 3.81 mph at 15, 30 and 60 minutes). Wu et al. [11] learn an adaptive dependency matrix through node embeddings and use stacked dilated one-dimensional convolutions in Graph WaveNet. Graph attention networks [10] learn which neighbours matter by attention and are the attention-based option named in the project brief.

2.3 Published reference points

Table 2 lists the METR-LA results reported by Li et al. [9] next to ours. They are shown for orientation only: the protocols differ (Section 4.2), so the numbers are not directly comparable, and reproducing the published DCRNN under our protocol is the first goal of the next phase.

2.4 Signal control and simulation

The brief also names optional simulated signal control. Simulators such as CityFlow [13] are built for reinforcement-learning research on signal plans. (The brief cites this work as “Zheng et al.”; the first author in the proceedings is Huichu Zhang.) Everything in this project concerning signal control is simulation only; no claim of live traffic control is made.

2.5 Research gap and hypotheses

Published graph models are usually compared with older or weaker baselines and under window-level splits. We therefore ask a narrower, testable question: with a leak-free protocol and strong per-sensor baselines, *where* does an independent model fail, and how much of that is recoverable by giving it neighbour information? Four hypotheses follow from the literature and from the propagation analysis of Section 3.3:

H1 The error of the best independent model grows with the forecast horizon, because the relevant information is not in the sensor’s own history.

Reference	Contribution (as stated by the authors)	Role in this project
Li et al. [9]	Diffusion on a directed graph, bidirectional random walks, encoder–decoder with scheduled sampling	Data and graph source, protocol to compare with, model to reproduce next; motivates the upstream/downstream experiment
Veličković et al. [10]	Attention over graph neighbours	The attention model named in the brief (next phase)
Yu et al. [12]	Traffic forecasting on graphs with fully convolutional structures	Alternative graph baseline
Wu et al. [11]	Adaptive dependency matrix, dilated convolutions	Ablation: fixed versus learned adjacency
Chen and Guestrin [2], Breiman [1], Hochreiter and Schmidhuber [3], Hoerl and Kennard [4]	Gradient boosting, random forests, LSTM, ridge regression	The four learned baselines
Kingma and Ba [8]	Adam optimiser	Training of the LSTM
Kaufman et al. [7]	Formulation and detection of leakage	Motivates the purge gap and the leakage tests
Hyndman and Koehler [5]	Properties of forecast-accuracy measures	Choice and caveats of MAE, RMSE and MAPE
Jagadish et al. [6]	Origin of the METR-LA data (as cited in 9)	Dataset
Zhang et al. [13]	Reinforcement-learning simulator for traffic signals	Optional simulation-only signal control

Table 1: Research references and their role in this project.

H2 Its error is concentrated right after congestion onsets, where propagation matters most.

H3 Residuals of road-linked sensors are correlated while those of unconnected sensors are not.

H4 Giving the model neighbour information lowers its error, downstream neighbours more than upstream ones, but one-hop hand-made features do not remove the gap.

Section 8.1 returns to them once the results are known.

3 Data: METR-LA

METR-LA contains speed readings (mph) from loop detectors on Los Angeles County freeways, collected from March to June 2012; we use the version distributed with the DCRNN reference implementation [9], whose data originate from Jagadish et al. [6]. The sensor graph is the supplied directed adjacency matrix, a thresholded Gaussian kernel of road-network distance. The order of the speed columns was verified against the adjacency order; this is enforced in code and tested.

3.1 Missing data

A reading of 0 mph means that the sensor was down, not that traffic stopped (Table 3). Of the 8.11% zero readings, 6.27 percentage points are whole-network outages (2 148 timestamps at

Published (Li et al.)	15 min	30 min	60 min	This work: 15 / 30 / 60
Historical average (HA) [9]	4.16	4.16	4.16	4.42 / 4.42 / 4.42
FC-LSTM [9]	3.44	3.77	4.37	2.97 / 3.55 / 4.35
DCRNN [9]	2.77	3.15	3.60	not yet built
XGBoost (this work)	–	–	–	3.02 / 3.51 / 4.01

Table 2: METR-LA mean absolute error (mph) at 15, 30 and 60 minutes as reported by Li et al. [9] (Table 1 of that paper), and ours on the strict test block. Protocols differ; see the text.

Property	Value
Sensors and readings	207 loop detectors; 34 272 timestamps each
Period	2012-03-01 to 2012-06-27, 5-minute clock, no gaps
Readings equal to 0 mph	8.11% (6.27 points whole-network, 1.84 points single-sensor)
Whole-network outages	2 148 timestamps; longest 31.8 h
Weekday free-flow speed	63.2 mph (weekend minimum 56.7 mph)
Weekday dips below 90% of free flow	07:05–10:45 (low 49.0 mph); 14:30–19:15 (low 45.7 mph)
Speed distribution	median 63.22 mph; 66.36% at least 60 mph; 10.5% below 40 mph; 2.75% below 20 mph
Road graph	207 nodes; 1 515 directed links; mean degree 7.32; 26.67% two-way; 1 isolated sensor

Table 3: Dataset facts (computed in the exploratory analysis, `results/phase1_findings.json`).

which all sensors are zero, up to 31.8 hours), i.e. logging failures, and only 1.84 points are single-sensor failures (Figure 1). Treating zeros as traffic would teach a model that roads randomly stop, so every later stage masks them.

3.2 Daily rhythm

Weekday speeds dip in the morning and, more deeply, in the afternoon, while weekend speeds stay near free flow (Figure 2). The brief's peak definition (weekdays 07:00–09:59 and 16:00–19:59) lies inside the detected dips. Because 66.36% of readings are at least 60 mph, average errors are dominated by easy free-flow periods; this motivates reporting peak and congestion-onset results separately (Figure 3).

3.3 The road graph and propagation of congestion

The graph has 1 515 directed links, of which only 26.67% are two-way, so direction carries information (Figure 4). To test whether neighbouring sensors carry predictive information, each sensor's typical (weekday-or-weekend and time-slot) pattern was subtracted and the remaining disturbance was cross-correlated along every directed link, using training data only. 62.2% of links are weak (peak correlation below 0.3). Among the 499 links with a clear peak, the downstream sensor moves first on 156 links and the upstream sensor first on 41, and the implied backward wave speed is about 16 km/h (Figure 5). A jam therefore appears at the downstream neighbour before the upstream sensor, which a model that sees only its own history cannot anticipate.

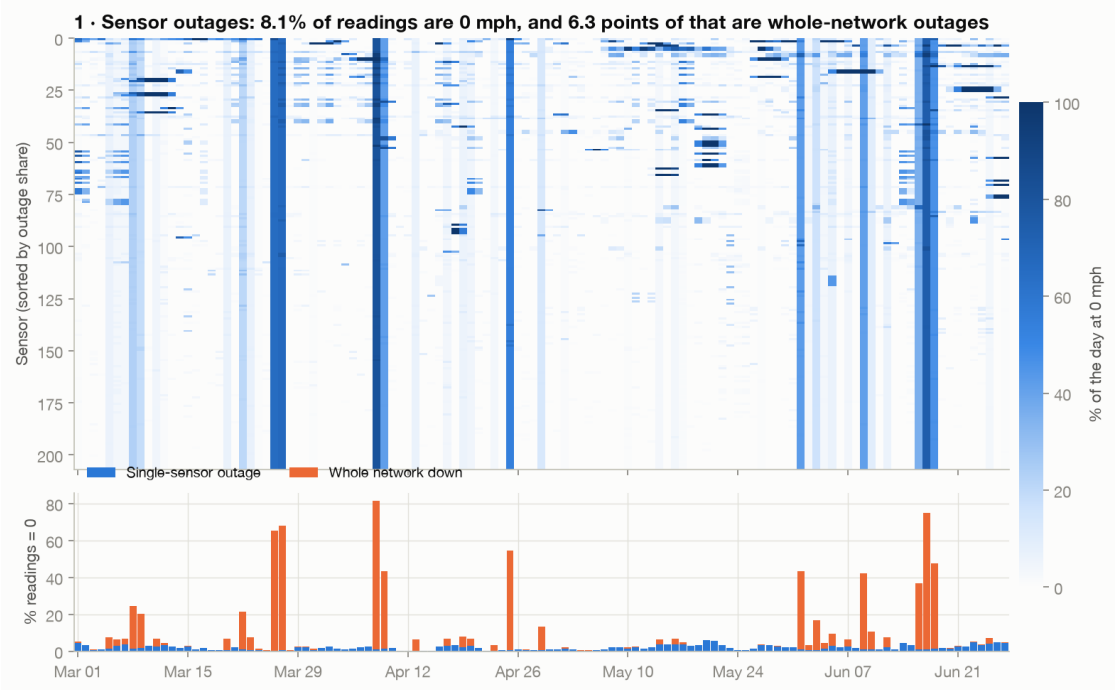


Figure 1: Share of readings equal to zero by sensor and day, and the part explained by whole-network outages.

4 Methodology

4.1 Problem formulation

Let $x_t^{(i)}$ be the speed of sensor $i \in \{1, \dots, 207\}$ at time step t (5 minutes) and $m_t^{(i)} \in \{0, 1\}$ indicate a real reading. Given the last 12 steps $x_{t-11:t}^{(i)}$ (60 minutes) and time features, the task is to forecast $\hat{y}_{t+h}^{(i)}$ for $h = 1, \dots, 12$ (5 to 60 minutes ahead). Headline results use $h = 3, 6, 12$ (15, 30 and 60 minutes). All models use exactly this 12-step history.

4.2 Leak-free preprocessing

Chronological split with a purge gap. Timestamps are split 70, 10 and 20 percent into train, validation and test blocks. A window is kept only if all 12 of its target timestamps lie in one block; windows whose targets straddle a boundary are purged (11 per boundary), so no target timestamp is in two splits. Inputs may look back into earlier data: that is the past, not a label, and matches deployment. Table 5 shows why the purge matters. Splitting *windows* instead

Split	Windows	Rows	First target	Last target	Masked targets
Train	22 933	4 747 131	2012-03-01 01:00	2012-05-23 07:05	2.96%
Val	3 417	707 319	2012-05-23 07:10	2012-06-04 04:45	6.98%
Test	6 843	1 416 501	2012-06-04 04:50	2012-06-27 23:55	12.12%

Table 4: Strict split: windows, rows (windows × 207 sensors), target period and share of masked targets. The training block also excludes 1 034 windows whose targets are over 90% masked.

of timestamps, as in the reference DCRNN script, makes the first validation target earlier than the last training target, i.e. train and validation targets overlap by 50 minutes; such overlap is a form of the leakage studied by Kaufman et al. [7]. Both protocols are implemented so published

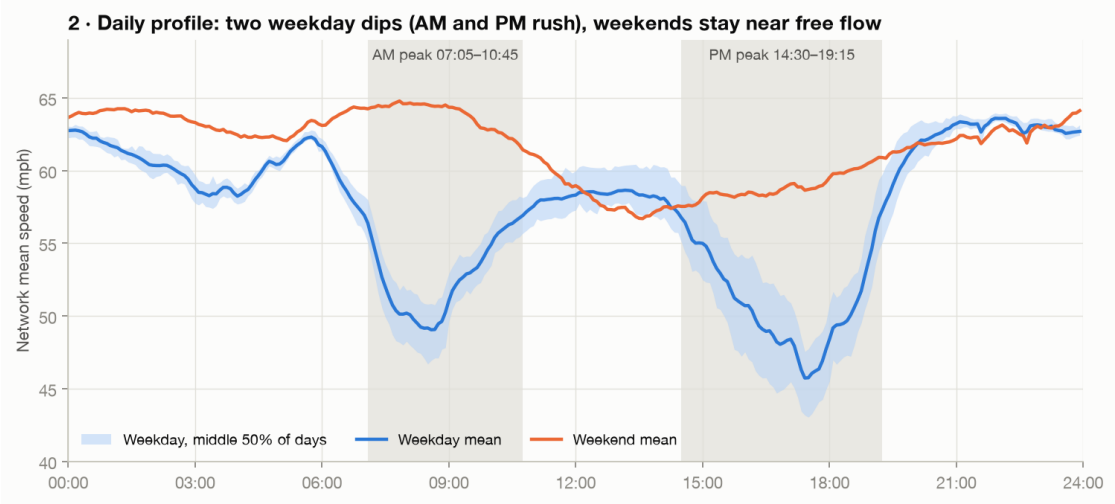


Figure 2: Network-mean speed by time of day for weekdays and weekends (training period only).

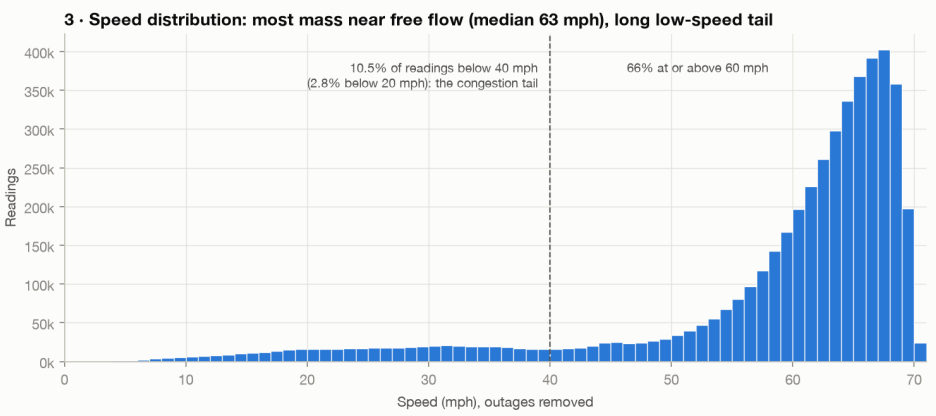


Figure 3: Distribution of speed readings (outages excluded): most readings are near free flow, and congested readings are a minority.

numbers can be compared later; all results here use the strict protocol, and a test proves the overlap exists in the reference protocol and not in ours.

	Strict split (used)	Reference window split
Split unit	timestamps, then windows	windows
Last train target	2012-05-23 07:05	2012-05-23 07:40
First validation target	2012-05-23 07:10	**2012-05-23 06:50** (earlier)
Scaler mean / std (mph)	58.58 / 12.82, outages excluded	54.41 / 19.49, 0 mph included

Table 5: Strict timestamp split versus the reference window split.

Outage masking. Zeros become missing. In the inputs, a gap is forward-filled from the last real reading inside the same 12-step window, otherwise from the training mean of that sensor and time-of-day slot, and a mask channel tells the model the value is imputed. Targets stay missing; every loss and metric ignores them. Training windows whose targets are over 90% masked, mostly whole-network outages, are dropped; validation and test keep every window.

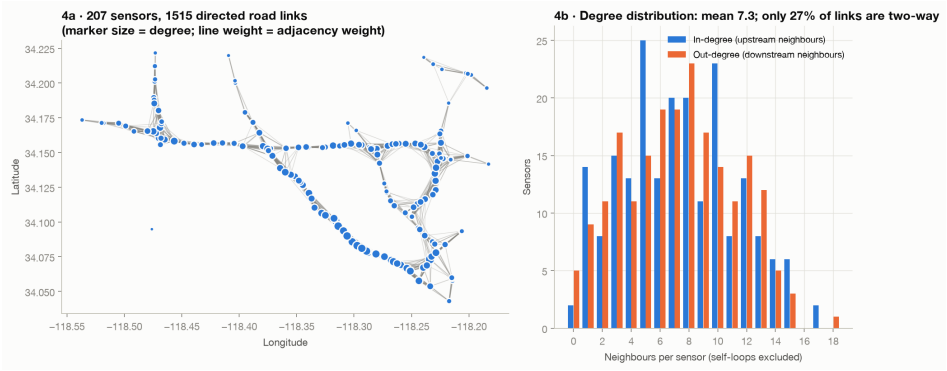


Figure 4: The sensor graph over Los Angeles (directed links, node size by degree).

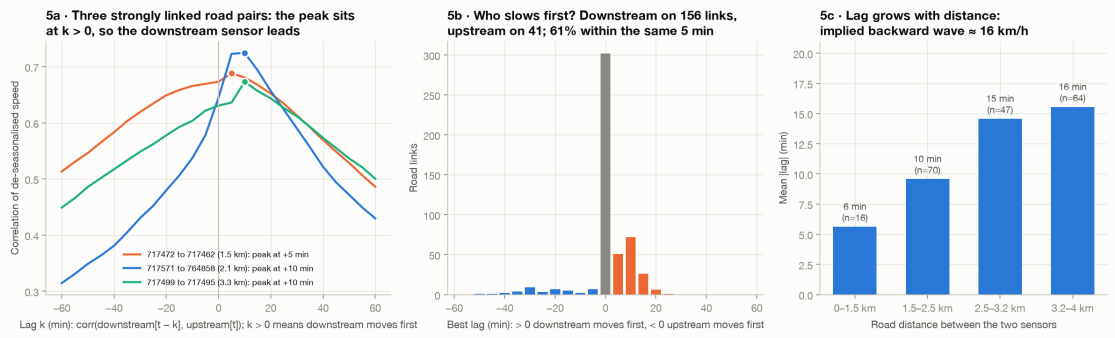


Figure 5: Lead–lag analysis along directed links: most strongly coupled links show the downstream sensor moving first, with the lag growing with distance.

Scaling. Speeds are standardised as $z = (x - \mu)/\sigma$ with $\mu = 58.58$ mph and $\sigma = 12.82$ mph, computed from training timestamps only with outages excluded. Predictions are inverse-transformed so that all errors are in mph.

Time features and adjacency. Time of day and day of week enter as sine and cosine pairs plus a weekend flag, so that 23:55 and 00:05 are neighbours. The adjacency is rebuilt from road distances d_{ij} as $W_{ij} = \exp(-(d_{ij}/\sigma_d)^2)$ if this is at least $\kappa = 0.1$ and 0 otherwise, with σ_d the standard deviation of the finite distances, and is asserted equal to the supplied matrix; it stays directed and fixed.

4.3 Feature table

One row is one sensor at one forecast origin: 12 lagged speeds; mean, standard deviation and minimum over the last 3, 6 and 12 steps; two differences (speed now minus 1 and 3 steps earlier); 12 real-reading flags and the count of imputed lags; five time features; and five segment descriptors computed on training data only (sensor id, mean speed, free-flow speed as the 85th percentile, in-degree and out-degree), 46 columns in total. The split sizes in Table 4 give 4 747 131 training, 707 319 validation and 1 416 501 test rows. Flow and occupancy, which the brief lists, are not recorded in METR-LA; they require PeMS data and are a data limitation rather than omitted work.

4.4 Models

Table 6 lists the six baselines. XGBoost and the Random Forest are *independent* models: a row contains only one sensor's own history, time and static descriptors and never a neighbour's speed.

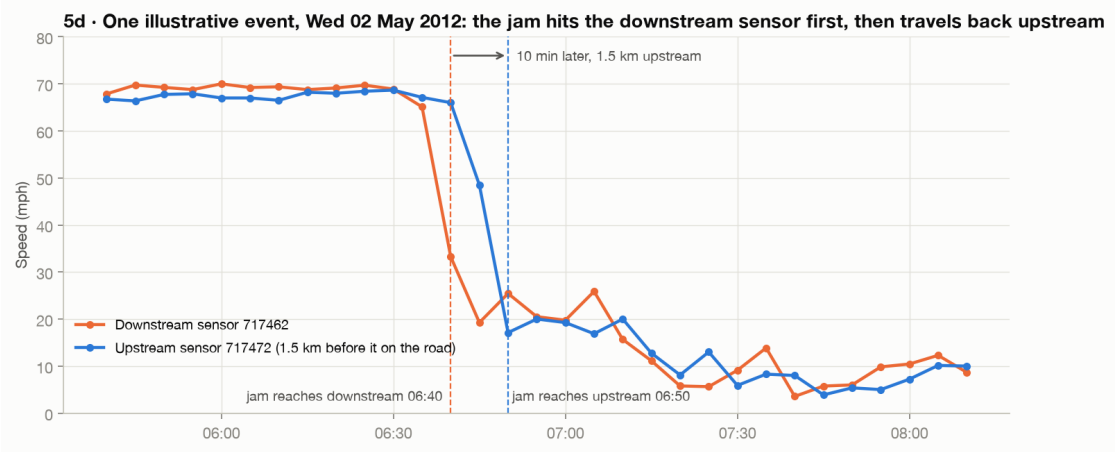


Figure 6: An illustrative event (2 May 2012): the downstream sensor drops to about 20 mph at 06:40 while the upstream sensor is still at about 66 mph. The event was selected by a rule; the evidence is the all-link statistics above.

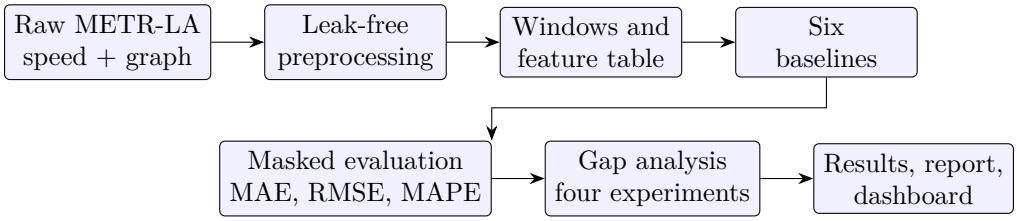


Figure 7: Pipeline overview. Every stage is a command in the project's Makefile.

The LSTM shares weights across sensors but also sees no neighbour, so it separates the effect of deep learning from the effect of the graph; Adam [8] was used for training. All hyper-parameters and early stopping use the validation block only.

4.5 Evaluation protocol

Errors are computed only where the target is a real reading:

$$\mathrm{MAE} = \frac{\sum m\,|\hat{y} - y|}{\sum m}, \quad \mathrm{RMSE} = \sqrt{\frac{\sum m\,(\hat{y} - y)^2}{\sum m}}, \quad \mathrm{MAPE} = 100\,\frac{\sum m\,|\hat{y} - y| / \max(y, 1)}{\sum m},$$

with the sums over test windows, sensors and (for a given horizon) the 12-step output. MAPE is sensitive to small denominators [5]; the 1 mph floor only guards a pathological reading, as targets are real speeds. To summarise forecasts in plain terms we also report *percent accuracy* (100 minus MAPE) and the share of forecasts within 5 mph of the true speed. Peak is weekdays 07:00–09:59 and 16:00–19:59, judged on the timestamp being predicted. Test numbers are written to the results files but never used to choose anything.

4.6 Gap-analysis methods

Congestion onset. An onset is a speed drop of more than 20 mph within 15 minutes starting from at least 50 mph (3-step rolling median). A target is "after an onset" if it lies within 30 minutes after one at that sensor; this labels evaluation points only and is never a model input.

Residual correlation. For each pair of sensors the correlation of their XGBoost residual series, using only timestamps where both readings are real, is grouped by the number of road

Rung	Model	Configuration
0	Persistence	Repeats the last (imputed) speed for all 12 horizons.
1	Historical average	Training mean per sensor, weekday-or-weekend and 5-minute slot.
2	Ridge per sensor	One linear model per sensor and horizon (207×12) on the 41 time-varying features, standardised per sensor. Penalty chosen from $\{1, 10, 100, 1000\}$ on validation: 1000 (validation MAE 3.859 to 3.858, so the choice barely matters).
3	XGBoost	One regressor per horizon trained on all sensors stacked, sensor id as a feature, masked targets given weight 0. Maximum depth searched at the 30-minute horizon: 10 gives 3.291, 8 gives 3.342, 6 gives 3.413 validation MAE; depth 10 chosen. Learning rate 0.05, row and column subsample 0.8, up to 600 rounds with early stopping (50) on validation; every 2nd window used.
4	Random Forest	One multi-output forest predicting all 12 horizons: 100 trees, depth 20, minimum leaf 10, 30% bootstrap samples, 50% of features per split. Trained on every 6th window keeping rows whose 12 targets are all observed (742 613 rows); evaluated on every test row with the mask.
5	LSTM, no graph	Two LSTM layers of 64 units, dropout 0.1, one set of weights shared by all 207 sensors; input is each sensor's own 12×7 window, output 12 horizons. Masked MAE in mph, Adam (learning rate 0.002, halved on plateau), gradient clipping 5, at most 12 epochs. All 12 epochs were used; best validation MAE 3.208.

Table 6: The six baselines and their configuration.

links between them.

Neighbour-information test. XGBoost is given summaries of a sensor's road neighbours in one or both directions ("upstream" are sensors with a link into it, "downstream" those it links to): the weighted mean neighbour speed now and 5, 15 and 30 minutes ago, the slowest neighbour now, and the largest 15-minute drop among neighbours, i.e. 6 features per direction. The same depth, row thinning and early stopping as the base model are used with no extra tuning, for the 15, 30 and 60 minute horizons only.

5 Results: forecasting baselines

5.1 Scores on the test block

Table 7 reports all three metrics at the three headline horizons; Appendix A gives every horizon and Appendix B the validation scores. XGBoost has the lowest RMSE at all three horizons

Model	MAE 15	MAE 30	MAE 60	RMSE 15	RMSE 30	RMSE 60	MAPE 15	MAPE 30	MAPE 60	Pooled
Persistence	3.49	4.20	5.37	6.39	8.01	10.29	8.6	10.8	14.5	4.23
Historical average	4.42	4.42	4.42	8.08	8.08	8.08	13.9	13.9	13.9	4.42
Ridge per sensor	3.31	4.08	5.17	5.88	7.19	8.67	8.9	11.7	15.5	4.08
Random Forest	3.08	3.68	4.45	5.61	6.71	7.85	8.3	10.6	13.5	3.64
LSTM (no graph)	**2.97**	3.55	4.35	5.84	7.19	8.84	**8.0**	10.3	13.5	3.54
XGBoost	3.02	**3.51**	**4.01**	**5.47**	**6.41**	**7.27**	8.2	**10.1**	**12.0**	3.44

Table 7: Test scores (MAE and RMSE in mph, MAPE in percent) at 15, 30 and 60 minutes; best in each column in bold. "Pooled MAE" averages all 12 horizons.

and the lowest MAE and MAPE at 30 and 60 minutes; its MAE is 3.02, 3.51 and 4.01 mph,

which is 14% lower than persistence at 15 minutes and 25% lower at 60 minutes, and it has the best pooled MAE (3.44). The LSTM has the lowest MAE and MAPE at 15 minutes (2.97 against 3.02 mph), a difference of 0.04 mph from a single training run that should not be over-interpreted. Persistence is a strong floor at 15 minutes but its MAE grows from 3.49 to 5.37 mph by 60 minutes. The historical average ignores the present, so its error is flat (4.42 mph); it beats persistence and Ridge at 60 minutes but not the tree models. Ridge is competitive at 15 minutes, but its MAPE at 60 minutes (15.5%) is the worst of the six because a linear model extrapolates the recent trend too far.

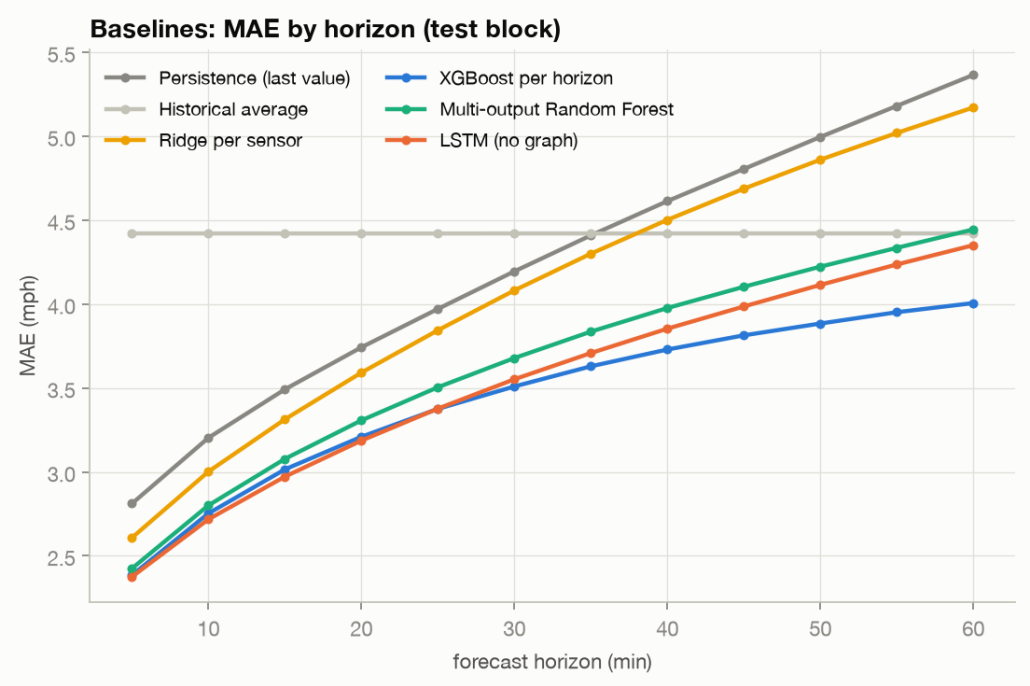


Figure 8: Test MAE by forecast horizon for all six baselines.

5.2 Accuracy in plain terms

Table 8 converts the errors to percentages. Speed forecasting is a regression task, so there is no single accuracy figure; percent accuracy and the within-5-mph share are summaries, not formal metrics. The best independent model is about 91.8% accurate at 15 minutes, 89.9% at 30 and 88.0% at 60, and 84.3% of its 15-minute forecasts are within 5 mph. These percentages look high mainly because traffic is free-flowing most of the time (even persistence scores about 91% at 15 minutes), so the error in mph and the error during congestion are the more informative quantities.

5.3 Peak versus off-peak

Peak-hour error is 1.42 to 1.95 times the off-peak error across all models and horizons, and the penalty grows with the horizon (Table 9). XGBoost's weekday-peak MAE rises from 4.22 to 6.23 mph, while its off-peak MAE rises from 2.69 to 3.32 mph. Rush hour is when congestion forms and travels, so it is where a network-aware model has the most to gain.

Model	Acc. 15	Acc. 30	Acc. 60	≤5 mph 15	≤5 mph 30	≤5 mph 60
Persistence	91.4	89.2	85.5	80.8	77.9	73.5
Historical average	86.1	86.1	86.1	76.5	76.5	76.5
Ridge per sensor	91.1	88.3	84.5	82.3	77.9	70.6
Random Forest	91.7	89.4	86.5	84.1	80.6	75.9
LSTM (no graph)	92.0	89.7	86.5	85.1	82.6	79.5
XGBoost	91.8	89.9	88.0	84.3	81.4	78.6
XGBoost + neighbours	92.3	90.4	88.3	84.7	81.7	78.8

Table 8: Test percent accuracy ("Acc.", 100 minus MAPE) and share of forecasts within 5 mph, at 15, 30 and 60 minutes.

Model	Weekday peak	Weekday off-peak	Weekend
Persistence	4.61 / 6.05 / 8.32	3.25 / 3.74 / 4.62	2.86 / 3.31 / 3.98
Historical average	6.82 / 6.82 / 6.82	3.58 / 3.58 / 3.58	3.94 / 3.94 / 3.94
Ridge per sensor	4.58 / 6.02 / 7.93	3.00 / 3.56 / 4.40	2.71 / 3.27 / 4.10
Random Forest	4.31 / 5.51 / 7.08	2.75 / 3.15 / 3.63	2.56 / 3.02 / 3.65
LSTM (no graph)	4.16 / 5.34 / 6.96	2.69 / 3.09 / 3.66	2.39 / 2.75 / 3.23
XGBoost	4.22 / 5.22 / 6.23	2.69 / 3.01 / 3.32	2.50 / 2.90 / 3.34

Table 9: Test MAE (mph) at 15 / 30 / 60 minutes by period.

5.4 What XGBoost relies on

Table 10 and Figure 9 show the share of total gain by feature. The model leans on the current speed and its short-term mean at every horizon (current speed: 54.0% of gain at 15 minutes, 36.2% at 60), and as the horizon grows the clock and the sensor's typical speed gain weight. Nothing in the feature set describes what is happening elsewhere on the network.

Feature	15 min	30 min	60 min
speed_t	54.0%	47.4%	36.2%
mean_3	29.7%	26.9%	21.9%
speed_t-1	3.8%	3.4%	2.5%
mean_6	2.6%	3.0%	2.7%
cos_tod	1.3%	3.0%	7.1%
sensor_mean	1.1%	2.7%	6.6%
mean_12	0.9%	–	–
sin_tod	0.9%	2.1%	4.8%
free_flow_p85	–	1.2%	2.5%

Table 10: Share of XGBoost's total gain by feature (top eight per horizon; "–" means outside the top eight).

6 Results: identifying the gap

Section 3.3 showed that congestion propagates between sensors; this section tests whether the best independent model, XGBoost, fails exactly where that matters. All results use the test block and the saved XGBoost predictions.

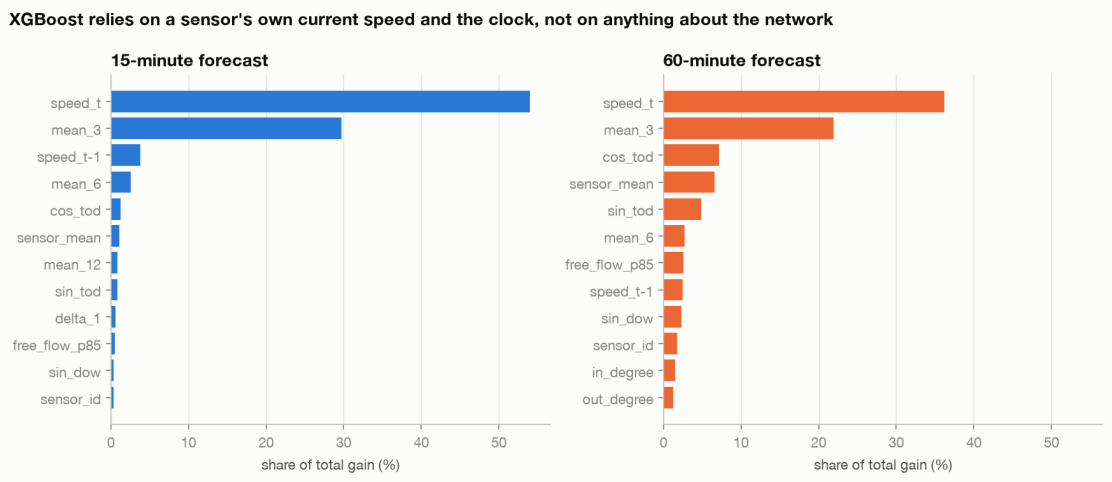


Figure 9: XGBoost feature importance at 15 and 60 minutes.

6.1 Error grows steadily with horizon

XGBoost's MAE rises from 3.02 to 3.51 to 4.01 mph and its RMSE from 5.47 to 6.41 to 7.27 mph (Figure 10). Beyond about 15 minutes the jam that will arrive is not yet visible in the sensor's own history; it is at a neighbour.

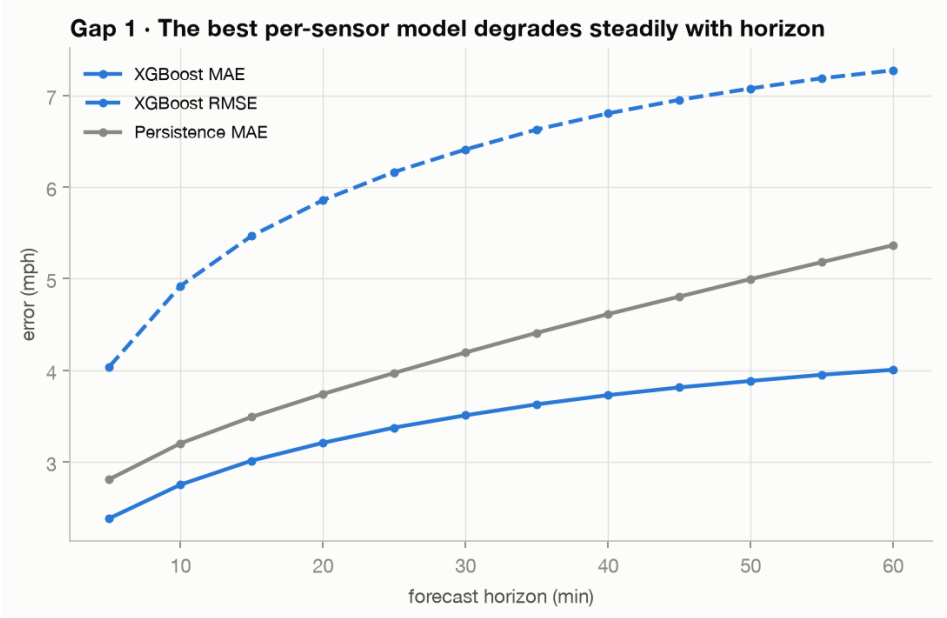


Figure 10: XGBoost error against forecast horizon, with persistence for reference.

6.2 Error after congestion onsets

The test period contains 3 911 onsets; the affected targets are 2.17% of all real targets. In the 30 minutes after an onset, XGBoost's MAE is 16.2 mph at 15 minutes against 2.7 mph at other times, a ratio of 5.9; the ratios at 30 and 60 minutes are 6.4 and 5.2 (Table 11, Figure 11). The model keeps forecasting free flow until the slowdown appears in its own lags. Persistence fails the same way and worse, so the cause is the information available and not the learner.

Model	Horizon	After onset	Other times	Ratio
XGBoost	15 min	16.20	2.72	5.9×
XGBoost	30 min	20.12	3.14	6.4×
XGBoost	60 min	19.09	3.67	5.2×
Persistence	15 min	17.90	3.17	5.6×
Persistence	30 min	25.35	3.73	6.8×
Persistence	60 min	27.15	4.88	5.6×
XGBoost + neighbours	15 min	14.42	2.65	5.4×
XGBoost + neighbours	30 min	18.77	3.08	6.1×
XGBoost + neighbours	60 min	18.89	3.62	5.2×

Table 11: Test MAE (mph) in the 30 minutes after a congestion onset versus other times.

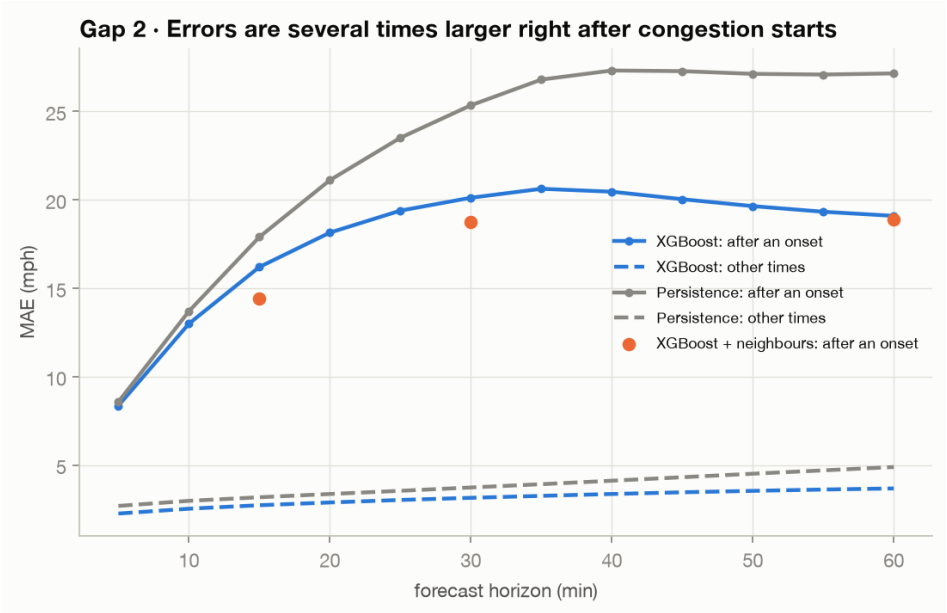


Figure 11: MAE after congestion onsets versus other times, by horizon.

6.3 Errors of linked sensors move together

Table 12 and Figure 12 show that the correlation of XGBoost residuals depends on road distance: 0.20 for sensors one link apart at 60 minutes, 0.05 for two links and about zero for unconnected sensors. A network-wide cause such as time of day would also correlate unconnected sensors; it does not. The correlation grows with horizon (0.12 at 15 minutes to 0.20 at 60), as expected if the missing information is congestion travelling between sensors. This is an association, not proof of causation.

6.4 Neighbour-information test

Table 13 and Figure 13 give the key experiment. Every variant beats the independent model at every horizon, and validation shows the same ordering (both directions: −4.2%, −3.8% and −2.5% MAE). With both directions, test MAE falls to 2.91, 3.42 and 3.95 mph, i.e. by 3.5%, 2.6% and 1.4%, and RMSE at 15 minutes from 5.47 to 5.17 mph. Downstream neighbours help more than upstream ones, consistent with the finding that congestion is seen first downstream. Right after congestion onsets the MAE at 15 minutes falls from 16.20 to 14.42 mph (11%) and at 30 minutes by 7%, a much larger relative gain than the average.

Pair distance	XGB 15 min	XGB 30 min	XGB 60 min	XGB + nb. 60 min
1 link	0.115	0.158	0.200	0.197
2 links	0.019	0.031	0.050	0.044
3 links	0.010	0.014	0.025	0.020
4+ links	0.007	0.010	0.017	0.014
not connected	0.003	0.005	-0.002	0.000

Table 12: Mean correlation of residuals between sensor pairs by number of road links (test block). The last column is the neighbour-feature model of Section 6.4.

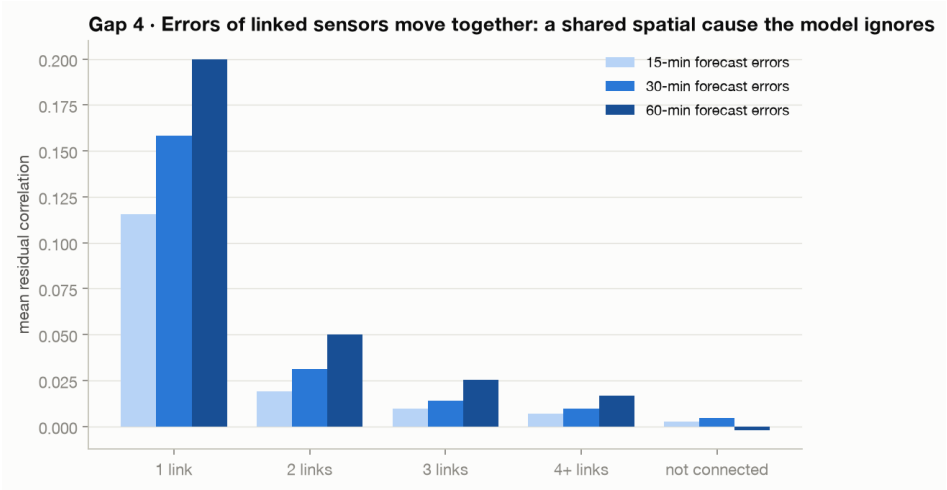


Figure 12: Residual correlation by road-graph distance.

Why this is not enough. The 12 neighbour features together receive only 4.5%, 6.3% and 7.8% of the model's gain at 15, 30 and 60 minutes. The gain shrinks at 60 minutes, the residual correlation of linked sensors barely moves (0.200 to 0.197 at 60 minutes), and the error after onsets remains several times normal. A tree model with one-hop summaries cannot learn which neighbour matters when, nor look several links away; this is what attention over the road graph [10] and multi-hop diffusion [9] are designed to learn.

6.5 Gap statement

Independent segment models forecast each sensor from its own history. Congestion forms at bottlenecks and travels backwards along road links at about 16 km/h, so at longer horizons the most predictive information sits at neighbouring sensors. We observe this as error rising steadily with horizon, errors 5.9 to 6.4 times larger in the 30 minutes after a congestion onset, and residuals correlated only between road-linked sensors. Handing XGBoost one-hop neighbour summaries lowers error at every horizon but leaves linked residuals correlated and onset errors several times normal. A graph model that learns which neighbours matter, over several hops, should close this gap.

Model	MAE 15	MAE 30	MAE 60	RMSE 15	MAPE 15
XGBoost, own history only	3.015	3.510	4.006	5.47	8.2
+ upstream neighbours	2.971 (-1.5%)	3.472 (-1.1%)	3.978 (-0.7%)	5.37	8.0
+ downstream neighbours	2.945 (-2.3%)	3.461 (-1.4%)	3.971 (-0.9%)	5.26	7.8
+ both directions	2.908 (-3.5%)	3.418 (-2.6%)	3.949 (-1.4%)	5.17	7.7

Table 13: Test MAE (mph, change versus own-history XGBoost in brackets) with neighbour features; RMSE (mph) and MAPE (%) at 15 minutes.

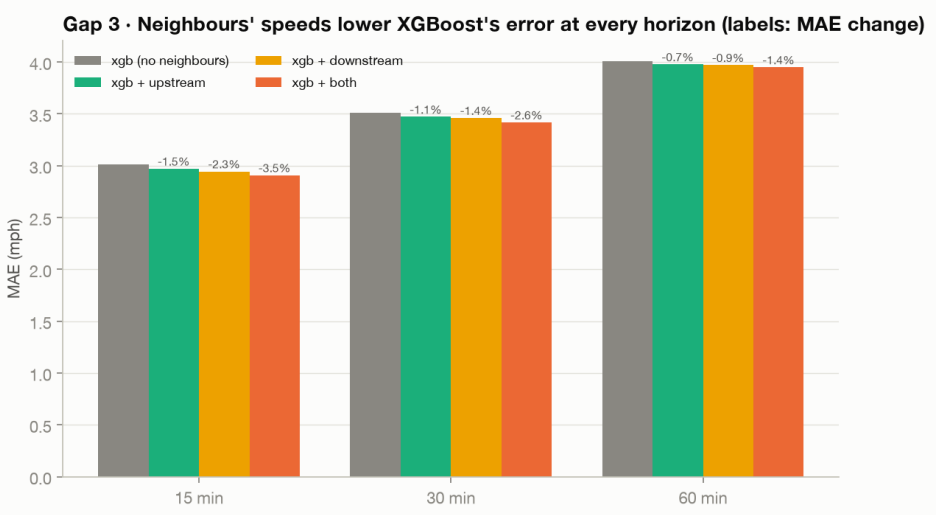


Figure 13: Test MAE of XGBoost with no, upstream, downstream and both-direction neighbour features.

7 Software, reproducibility and deployment

7.1 Repository

The code is organised as a small Python package with a command for every stage (https://github.com/ManasSaxena14/TrafficFlowControl_AIML-DL_T036):

```
.
|-- app.py                  Streamlit dashboard (deployed)
|-- Makefile                one command per stage
|-- configs/data/           split, window, masking and peak definitions
|-- src/
|   |-- data/               loading, leak-free preprocessing, readers
|   |-- features/           46-column per-sensor feature table
|   |-- models/             baselines: simple, trees, LSTM
|   |-- eval/               masked metrics, peak evaluation, accuracy
|   |-- analysis/           exploratory analysis and gap analysis
|   |-- train/              runners for baselines and gap experiments
|   `-- viz/                figures
|-- tests/                  automated tests
|-- results/                every result table (CSV and JSON)
|-- notebooks/              exploratory data analysis
|-- scripts/                data download, report table generator
`-- docs/                   this report, figures, slides
```

7.2 Testing

Test file	Tests	What it guards
test_data.py	9	shapes, five-minute clock, sensor order equals graph order, adjacency properties, masking of outages
test_leakage.py	12	target order, no target in two splits, train-only scaler, no future inputs, masked metrics, adjacency rebuild
test_baselines.py	13	feature alignment, train-only statistics, baselines, tree models, peak labelling, accuracy
test_gap.py	6	neighbour direction and features, onset labelling, residual correlation
Total	**40**	all tests pass

Table 14: Automated tests.

The leakage tests check, among others, that target timestamps are ordered and never shared between splits, that the scaler depends only on training data, that altering readings after time t_0 does not change any window ending at or before t_0, and that masked points contribute exactly zero to the metrics.

7.3 Reproducing the results

`make setup data eda preprocess features baselines gap accuracy test` rebuilds everything from the raw download, and `make report` regenerates the tables in this document from the result files and compiles it. Intermediate data (about 2.5 GB per protocol) is not committed. The repository had 37 commits when this report was built.

7.4 Interactive dashboard

A Streamlit application, `app.py`, presents the saved results in five pages (overview, baseline models, model comparison, spatial gap analysis and an XGBoost deep dive). It reads only the pre-computed files in `results/` and does not train or import any model code, so it deploys on Streamlit Community Cloud with the repository's `requirements.txt`. The deployed dashboard is available at https://trafficflowamldl.streamlit.app.

8 Discussion and limitations

8.1 Findings in the light of the literature

The findings agree with the literature. Li et al. [9] argue that a bidirectional walk on a directed graph captures both upstream and downstream influence and show a gain over an undirected variant; in our data only 26.67% of links are two-way, and downstream neighbours help the independent model more than upstream ones, so the directed structure is informative and will be tested directly through a directed-versus-undirected ablation. Our independent baselines (historical average, Ridge, Random Forest, XGBoost, LSTM) are of a similar magnitude to the FC-LSTM and historical-average results published for METR-LA (Table 2), although the protocols differ, and the tree models give a stronger yardstick than the classical set used there. Finally, the overlap we measured between train and validation targets in the window-level reference split is an instance of the leakage described by Kaufman et al. [7], which is why the purge gap and twelve leakage tests are part of the pipeline.

Hypothesis	Evidence in this report	Outcome
H1	XGBoost MAE rises from 3.02 to 4.01 mph from 15 to 60 minutes (Section 6)	Supported
H2	Error after onsets is 5.9 to 6.4 times the error at other times (Table 11)	Supported
H3	Residual correlation 0.20 for one link against about zero for unconnected sensors (Table 12)	Supported
H4	Neighbour features lower MAE by up to 3.5%, downstream more than upstream; residual correlation of linked sensors barely changes (Table 13)	Supported, and the gap remains

Table 15: The four hypotheses of Section 2.5 and their outcome.

8.2 Limitations

Speed only. METR-LA has no flow or occupancy, so the brief's flow and occupancy features are not used; PeMS would supply them.

One city, one season. The data are freeway sensors in Los Angeles in 2012, so the results may not transfer to other networks or to signalised urban streets.

Single runs, no significance tests. Each model was trained once (seed 42). Small differences, such as the LSTM against XGBoost at 15 minutes, should not be over-interpreted; significance testing is planned.

Limited tuning budgets. XGBoost used at least 580 of its 600 allowed rounds at 9 of 12 horizons, so a larger budget might help slightly, and the LSTM used all 12 epochs with validation error still improving slowly. XGBoost and the Random Forest were trained on thinned windows, and the neighbour experiment covers three horizons.

Validation and test differ. The test period has 12.12% masked targets against 6.98% on validation (2.96% in training), so test and validation errors are not directly comparable; the test block was used for reporting only.

Definitions drive the gap analysis. Onsets use one threshold definition, and residual correlation shows association, not causation.

No reproduction of published graph models yet. Table 2 is not a like-for-like comparison. Reproducing the published DCRNN within about 5% under our protocol is the first gate of the next phase.

Simulation only. Signal control, if added, will be evaluated only in a simulator and will not be presented as live traffic control.

9 Conclusions and next steps

We built a leak-free pipeline and six baselines, and showed that XGBoost is the strongest independent model (test MAE 3.02, 3.51 and 4.01 mph at 15, 30 and 60 minutes). Four experiments show that its remaining error concentrates right after congestion starts, at rush hour, and in a form shared between road-linked sensors. Giving it its neighbours' speeds helps at every horizon, so the missing information is spatial, but hand-made one-hop features capture only part of it.

Next steps. (i) A spatio-temporal graph attention network and a DCRNN [9] over the fixed directed road graph, with ablations (graph versus no graph, directed versus undirected), evaluated on the same splits, windows, masks and metrics; the targets to beat are XGBoost's 3.02, 3.51 and 4.01 mph. (ii) A fair comparison with peak and off-peak breakdowns and significance tests over seeds. (iii) Congestion maps, an optional simulated signal-control experiment, and extensions such as PeMS flow and occupancy.

Contributions. Manas Saxena built the preprocessing pipeline, baselines, gap experiments, report and slides; Harsh Raj set up the project and the exploratory analysis; Sumit contributed the Streamlit dashboard.

References

[1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[2] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.

[3] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[4] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.

[5] Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.

[6] H. V. Jagadish, Johannes Gehrke, Alexandros Labrinidis, Yannis Papakonstantinou, Jignesh M. Patel, Raghu Ramakrishnan, and Cyrus Shahabi. Big data and its technical challenges. *Communications of the ACM*, 57(7):86–94, 2014. doi: 10.1145/2611567.

[7] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):15:1–15:21, 2012.

[8] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980.

[9] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1707.01926.

[10] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1710.10903.

[11] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, and Chengqi Zhang. Graph WaveNet for deep spatial-temporal graph modeling. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, 2019. arXiv:1906.00121.

[12] Bing Yu, Haoteng Yin, and Zhanxing Zhu. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI)*, 2018. arXiv:1709.04875.

[13] Huichu Zhang, Siyuan Feng, Chang Liu, Yaoyao Ding, Yichen Zhu, Zihan Zhou, Weinan Zhang, Yong Yu, Haiming Jin, and Zhenhui Li. CityFlow: A multi-agent reinforcement learning environment for large scale city traffic scenario. In *The World Wide Web Conference (WWW '19), demo track*, 2019. doi: 10.1145/3308558.3314139. arXiv:1905.05217.

A Test MAE at every horizon

Model	5	10	15	20	25	30	35	40	45	50	55	60
Persistence	2.81	3.20	3.49	3.74	3.97	4.20	4.41	4.61	4.81	5.00	5.18	5.37
Historical average	4.42	4.42	4.42	4.42	4.42	4.42	4.42	4.42	4.42	4.42	4.42	4.42
Ridge per sensor	2.61	3.00	3.31	3.59	3.84	4.08	4.30	4.50	4.69	4.86	5.02	5.17
Random Forest	2.42	2.80	3.08	3.31	3.50	3.68	3.84	3.98	4.10	4.22	4.34	4.45
LSTM (no graph)	2.38	2.72	2.97	3.19	3.38	3.55	3.71	3.85	3.99	4.11	4.24	4.35
XGBoost	2.38	2.75	3.02	3.21	3.38	3.51	3.63	3.73	3.81	3.88	3.95	4.01

Table 16: Test MAE (mph) at forecast horizons of 5 to 60 minutes (column headings in minutes).

B Validation scores

Model	MAE 15	MAE 30	MAE 60	RMSE 15	RMSE 30	RMSE 60	MAPE 15	MAPE 30	MAPE 60	Pooled
Persistence	3.22	3.82	4.79	6.02	7.52	9.59	7.8	9.8	13.0	3.85
Historical average	4.74	4.74	4.73	8.75	8.75	8.73	14.5	14.4	14.4	4.74
Ridge per sensor	3.11	3.85	4.91	5.58	6.83	8.26	8.4	11.0	14.6	3.86
Random Forest	2.86	3.42	4.14	5.31	6.36	7.46	7.7	9.8	12.5	3.39
LSTM (no graph)	**2.73**	**3.23**	3.89	5.49	6.77	8.22	**7.3**	9.4	12.1	3.21
XGBoost	2.82	3.29	**3.78**	**5.19**	**6.12**	**6.95**	7.5	**9.4**	**11.1**	3.23

Table 17: Validation scores in the same layout as Table 7. The validation block was used only to choose settings and for early stopping.

C Traceability

Every number in this report is generated by `scripts/make_report_tables.py` from the following files; the only values typed into that script are the published reference numbers of Table 2.

Content	Source in the repository
Dataset facts, missing data, graph, propagation	`results/phase1_findings.json`, `docs/figures/fig01`–`fig05d`
Split sizes, masked share, scalers	`data/processed/` (rebuilt by `make preprocess`), `configs/data/metr_la.yaml`
Model configuration	`results/baselines/*.json`, `src/models/`
Test and validation scores	`results/baselines.csv`, `results/baselines/*.csv`
Accuracy summaries	`results/accuracy.csv`
Peak versus off-peak	`results/baselines_peak.csv`
Feature importance	`results/baselines/xgb_importance.csv`
Onsets and residual correlation	`results/gap/onset_vs_normal.csv`, `results/gap/residual_correlation.csv`, `results/phase4_findings.json`
Neighbour experiment	`results/gap_results.csv`, `results/gap/xgb_neighbours*.csv`
Figures	`docs/figures/*.png`, produced by `src/viz/`